

目录

- TOC {toc}

我们的下一个并行模式是有序合并操作，它将两个排序列表合并生成一个合并排序列表。有序合并操作可以作为排序算法的构建模块，如我们将在第13章“排序”中看到的。有序合并操作也构成了现代MapReduce框架的基础。本章提出了一种并行有序合并算法，其中每个线程的输入数据是动态确定的。数据访问的动态特性使得利用局部性和分块技术来提高内存访问效率和性能变得具有挑战性。动态输入数据识别(dynamic input data identification)的原理同样适用于许多其他重要计算，例如集合交集和集合并集。我们提出了越来越复杂的缓冲区管理方案，以实现有序合并及其他动态确定其输入数据的操作的内存访问效率不断提高。

12.1 背景

一个有序合并函数将两个排序列表A和B合并成一个单一排序列表C。在本章中，我们假设排序列表存储在数组中。我们进一步假设每个数组元素都有一个键。在键上定义了一个称为#的顺序关系。例如，键可以是简单的整数值，而 $\leq$ 可以定义为这些整数值之间的常规小于或等于关系。在最简单的情况下，元素仅由键组成。

假设我们有两个元素 e_1 和 e_2 ，其键分别为 k_1 和 k_2 。在基于关系 $\leq$ 的排序列表中，如果 e_1 出现在 e_2 之前，则 $k_1 \leq k_2$ 。基于顺序关系R的合并函数接受两个排序输入数组A和B，它们分别有m和n个元素，m和n不必相等。数组A和B都基于顺序关系R进行排序。该函数生成一个输出排序数组C，包含m+n个元素。数组C由数组A和B中的所有输入元素组成，并且根据顺序关系R排序。

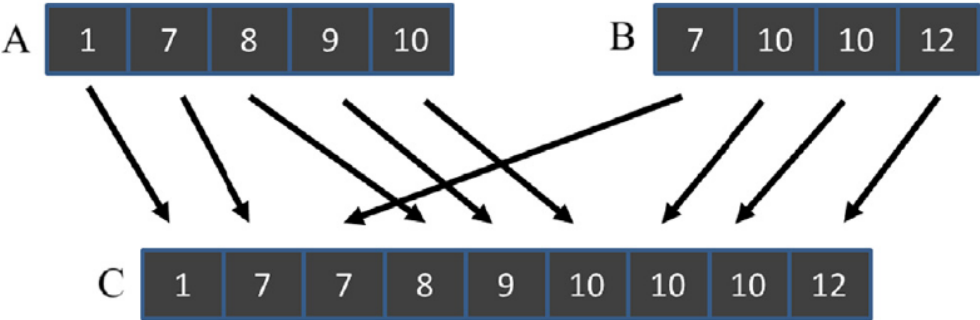


图12.1 合并操作示例。

图12.1展示了基于常规数值排序关系的简单合并函数的操作。数组A有五个元素($m=5$)，数组B有四个元素($n=4$)。合并函数生成包含A和B所有9个元素($m+n$)的数组C。这些元素必须排序。图12.1中的箭头显示了A和B的元素应该如何放置到C中以完成合并操作。每当A和B的元素之间的数值相等时，A的元素应首先出现在输出列表C中。这一要求确保了有序合并操作的稳定性。

一般来说，如果具有相同键值的元素在输出中的顺序与它们在输入中的顺序相同，则排序操作是稳定的。图12.1中的示例展示了合并操作的输入列表内和跨列表的稳定性。例如，两个值为10的元素从B复制到C时保持了它们的原始顺序。这说明了合并操作输入列表内的稳定性。另一个例子是值为7的A元素在C中位于值相同的B元素之前。这说明了合并操作跨输入列表的稳定性。稳定性属性允许排序操作保留当前排序操作所用键未捕获的先前排序。例如，列表A和B可能在按当前键进行排序之前已经根据不同的键进行了排序。保持合并操作的稳定性允许合并操作保留在先前步骤中完成的工作。

合并操作是归并排序的核心，这是一个重要的可并行化排序算法。如我们将在第13章“排序”中看到的，并行归并排序函数将输入列表分成多个部分并分发给并行线程。线程对各自的部分进行排序，然后协作合并排序部分。这种分而治之的方法允许高效的排序并行化。

在现代的MapReduce分布式计算框架中，如Hadoop，计算被分布到大量计算节点。reduce过程将这些计算节点的结果组装成最终结果。许多应用要求结果根据顺序关系进行排序。这些结果通常通过在归约树模式中使用合并操作来组装。因此，高效的合并操作对于这些框架的效率至关重要。

12.2 顺序合并算法

合并操作可以通过一个简单的顺序算法来实现。图12.2展示了一个顺序合并函数。

```
01 void merge_sequential(int *A, int m, int *B, int n, int *C) {
02     int i = 0; // Index into A
03     int j = 0; // Index into B
04     int k = 0; // Index into C
05     while ((i < m) && (j < n)) { // Handle start of A[] and B[]
06         if (A[i] <= B[j]) {
07             C[k++] = A[i++];
08         } else {
09             C[k++] = B[j++];
10         }
11     }
12     if (i == m) { // Done with A[], handle remaining B[]
13         while(j < n) {
14             C[k++] = B[j++];
15         }
16     } else { // Done with B[], handle remaining A[]
17         while(i < m) {
18             C[k++] = A[i++];
19         }
20     }
21 }
```

图12.2 串行顺序合并函数。

图12.2中的顺序函数由两个主要部分组成。第一部分是一个while循环（第05行），按顺序访问列表A和B的元素。循环从第一个元素开始：A[0]和B[0]。每次迭代填充输出数组C的一个位置；选择A的一个元素或B的一个元素填充该位置（第06至10行）。循环使用i和j来标识当前正在考虑的A和B元素；当首次进入循环时，i和j都为0。循环还使用k来标识当前要在输出列表数组C中填充的位置。在每次迭代中，如果元素A[i]小于或等于B[j]，则将A[i]的值赋给C[k]。在这种情况下，执行在进入下一次迭代前同时递增i和k。否则，将B[j]的值赋给C[k]。在这种情况下，执行在进入下一次迭代前同时递增j和k。

当到达数组A的末尾或数组B的末尾时，执行退出while循环。执行进入第二部分，如图12.2右侧所示。如果数组A已经被完全访问完（由i等于m表示），则代码将数组B的剩余元素复制到数组C的剩余位置（第13至15行）。否则，数组B已经被完全访问完，因此代码将数组A的剩余元素复制到C的剩余位置（第17至19行）。注意，为了保证正确性，if-else结构并不是必要的。我们可以简单地让两个while循环（第13至15行和第17至19行）跟随第一个while循环。根据第一个while循环是耗尽了A还是B，仅会进入其中一个while循环。然而，我们包含if-else结构以使代码对读者更直观。

我们可以用图12.1中的简单例子来说明顺序合并函数的操作。在while循环的前三次（第0至2次）迭代期间，A[0]、A[1]和B[0]分别被分配给C[0]、C[1]和C[2]。执行继续直到第5次迭代结束。在这一点上，列表A已经被完全访问，执行退出while循环。总共有六个C位置被A[0]至A[4]和B[0]填充。if结构的true分支中的循环用于将剩余的B元素（即B[1]至B[3]）复制到C的剩余位置。

顺序合并函数访问来自A和B的每个输入元素一次，并将它们写入到每个C位置一次。其算法复杂度为 $O(m + n)$ ，执行时间与要合并的元素总数成线性比例。

12.3 并行化方法

Siebert 和 Traff (2012) 提出了一种并行化合并操作的方法。在他们的方法中，每个线程首先确定它将生成的输出位置范围（输出范围），并使用该输出范围作为输入来调用协同排名函数（co-rank function），以识别出相应的输入范围，这些输入范围将合并以生成输出范围。一旦确定了输入和输出范围，每个线程就可以独立地访问它们的两个输入子数组和一个输出子数组。这种独立性使得每个线程可以在它们的子数组上执行顺序合并函数，从而并行进行合并。显然，所提出的并行化方法的关键是协同排名函数。现在我们来定义协同排名函数。

设A和B为两个输入数组，分别有m和n个元素。我们假设两个输入数组都是按照某种排序关系排序的。每个数组的索引从0开始。设C为通过合并A和B生成的排序输出数组。显然，C有 $m + n$ 个元素。我们可以做出以下观察：

观察1：对于任意k，使得 $0 \leq k < m + n$ ，有以下两种情况：情况1，有一个i，使得 $0 \leq i < m$ ，且在合并过程中C[k]的值来自A[i]；情况2，有一个j，使得 $0 \leq j < n$ ，且在合并过程中C[k]的值来自B[j]。

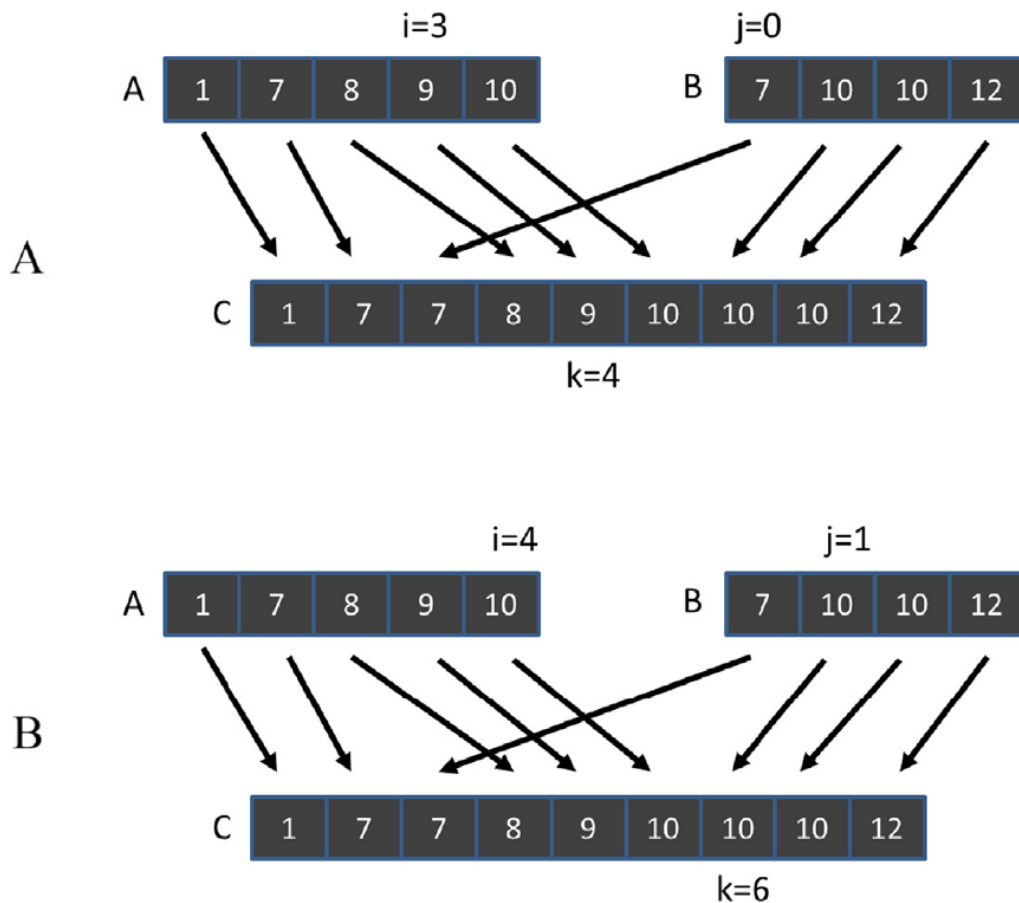


图12.3 观察1的例子。

图12.3展示了观察1的两种情况。在第一种情况下，C中有问题的元素来自数组A。例如，在图12.3A中，C[4]（值为9）来自A[3]。在这种情况下， $k=4$ ， $i=3$ 。我们可以看到，C[4]的前缀子数组C[0] - C[3]（四个元素的子数组）是合并A[3]的前缀子数组A[0] - A[2]（三个元素的子数组）和B[1]的前缀子数组B[0]（ $4 - 2 - 3 = 1$ 个元素的子数组）的结果。一般公式是，子数组C[0] - C[k - 1]（k个元素）是合并子数组A[0] - A[i - 1]（i个元素）和子数组B[0] - B[k - i - 1]（k - i个元素）的结果。

在第二种情况下，C中有问题的元素来自数组B。例如，在图12.3B中，C[6]的值来自B[1]。在这种情况下， $k=6$ ， $j=1$ 。C[6]的前缀子数组C[0] - C[5]（六个元素的子数组）是合并A[5]的前缀子数组A[0] - A[4]（五个元素的子数

组) 和 $B[1]$ 的前缀子数组 $B[0]$ (1 个元素的子数组) 的结果。此情况下的一般公式是, 子数组 $C[0] - C[k - 1]$ (k 个元素) 是合并子数组 $A[0] - A[k - j - 1]$ ($k - j$ 个元素) 和子数组 $B[0] - B[j - 1]$ (j 个元素) 的结果。

在第一种情况下, 我们找到 i 并推导出 j 为 $k - i$ 。在第二种情况下, 我们找到 j 并推导出 i 为 $k - j$ 。我们可以利用对称性并将这两种情况总结为一个观察:

观察2: 对于任意 k , 使得 $0 \leq k < m + n$, 我们可以找到 i 和 j , 使得 $k = i + j$, $0 \leq i < m$ 且 $0 \leq j < n$, 并且子数组 $C[0] - C[k - 1]$ 是合并子数组 $A[0] - A[i - 1]$ 和子数组 $B[0] - B[j - 1]$ 的结果。

Siebert 和 Traff (2012) 还证明了用于生成长度为 k 的 C 的前缀子数组所需的 A 和 B 的前缀子数组的 i 和 j 是唯一的。对于元素 $C[k]$, 索引 k 称为其排名。唯一的索引 i 和 j 称为其协同排名。例如, 在图 12.3A 中, $C[4]$ 的排名和协同排名分别是 4、3 和 1。另一个例子, $C[6]$ 的排名和协同排名分别是 6、5 和 1。

协同排名的概念为我们并行化合并函数提供了路径。我们可以通过将输出数组划分为子数组并将生成一个子数组的任务分配给每个线程来分配工作。一旦完成分配, 生成每个线程输出元素的排名就已知。每个线程然后使用协同排名函数来确定它需要合并到其输出子数组的两个输入子数组。

注意, 合并函数并行化与我们之前所有模式的并行化之间的主要区别在于, 每个线程使用的输入数据范围不能通过简单的索引计算来确定。每个线程使用的输入元素范围取决于实际输入值。这使得并行化合并操作成为一个有趣且具有挑战性的并行计算模式。

12.4 协同排名函数的实现

我们将协同排名函数定义为一个函数, 该函数接收输出数组 C 中某元素的排名 (k) 以及有关两个输入数组 A 和 B 的信息, 并返回对应的输入数组 A 中的协同排名值 (i)。协同排名函数的签名如下:

```
int co_rank(int k, int* A, int m, int* B, int n);
```

其中, k 是 C 中待确定的元素的排名, A 是指向输入数组 A 的指针, m 是 A 数组的大小, B 是指向输入数组 B 的指针, n 是输入 B 数组的大小, 返回值是 i , 即 A 中 k 的协同排名。调用者可以通过 $k - i$ 来推导出 B 中 k 的协同排名值 j 。

在研究协同排名函数的实现细节之前, 先了解并行合并函数使用该函数的方式是有益的。协同排名函数的使用如图 12.4 所示, 我们使用两个线程执行合并操作。假设线程 0 生成 $C[0] - C[3]$, 线程 1 生成 $C[4] - C[8]$ 。

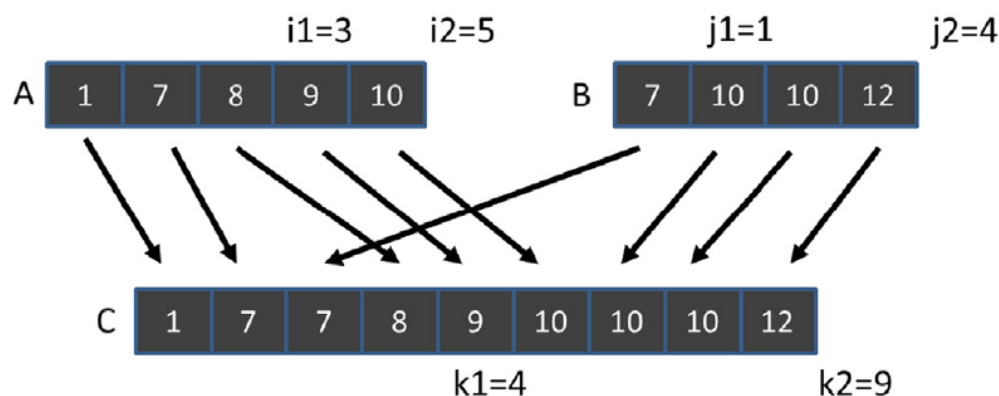


图 12.4 协同排名函数使用示例。

直观地说, 每个线程调用协同排名函数来确定将合并到分配给该线程的 C 子数组中的 A 和 B 子数组的起始位置。例如, 线程 1 使用参数 $(4, A, 5, B, 4)$ 调用协同排名函数。协同排名函数的目标是为线程 1 的排名值 $k1=4$ 识别协同

排名值 $i_1=3$ 和 $j_1=1$ 。也就是说，从 $C[4]$ 开始的子数组将通过合并从 $A[3]$ 和 $B[1]$ 开始的子数组生成。直观地说，我们正在寻找来自 A 和 B 的共计四个元素，这些元素将填充输出数组的前四个元素，而线程1将在这些元素之后合并其元素。通过目视检查，我们看到选择 $i_1=3$ 和 $j_1=1$ 满足我们的需求。线程0将取 $A[0] - A[2]$ 和 $B[0]$ ，留下 $A[3]$ （值为9）和 $B[1]$ （值为10），这正是线程1开始合并的位置。

如果我们将 i_1 的值更改为2，则需要将 j_1 的值设置为2，以便在线程1之前仍有共计四个元素。然而，这意味着我们将在线程0的元素中包含 $B[1]$ ，其值为10。这一值大于 $A[2]$ （值为8），这将包含在线程1的元素中。这样的更改会导致生成的 C 数组排序不正确。另一方面，如果我们将 i_1 的值更改为4，则需要将 j_1 的值设置为0，以保持元素总数为4。然而，这意味着我们将在线程0的元素中包含 $A[3]$ （值为9），其值大于 $B[0]$ （值为7），这将错误地包含在线程1的元素中。这两个例子表明可以通过搜索算法快速识别出正确的值。

另外，线程1不仅需要确定其输入段的起始位置，还需要确定它们的结束位置。因此，线程1还需使用参数（9, A, 5, B, 4）调用协同排名函数。从图12.4中可以看出，协同排名函数应产生协同排名值 $i_2=5$ 和 $j_2=4$ 。也就是说，因为 $C[9]$ 已经超过了 C 数组的最后一个元素，如果试图从 $C[9]$ 开始生成 C 子数组，则 A 和 B 数组的所有元素都应已用尽。一般来说，线程 t 要使用的输入子数组由线程 t 和线程 $t+1$ 的协同排名值确定： $A[i_t]$ 到 $A[i_{t+1}]$ 和 $B[j_t]$ 到 $B[j_{t+1}]$ 。

```

01  int co_rank(int k, int* A, int m, int* B, int n) {
02      int i = k < m ? k : m; // i = min(k,m)
03      int j = k - i;
04      int i_low = 0 > (k-n) ? 0 : k-n; // i_low = max(0,k-n)
05      int j_low = 0 > (k-m) ? 0 : k-m; // i_low = max(0,k-m)
06      int delta;
07      bool active = true;
08      while(active) {
09          if (i > 0 && j < n && A[i-1] > B[j]) {
10              delta = ((i - i_low + 1) >> 1); // ceil(i-i_low)/2)
11              j_low = j;
12              j = j + delta;
13              i = i - delta;
14          } else if (j > 0 && i < m && B[j-1] >= A[i]) {
15              delta = ((j - j_low + 1) >> 1);
16              i_low = i;
17              i = i + delta;
18              j = j - delta;
19          } else {
20              active = false;
21          }
22      }
23      return i;
24  }

```

图12.5 基于二分搜索的协同排名函数

协同排名函数本质上是一个搜索操作。由于两个输入数组都是已排序的，我们可以使用二分搜索甚至更高阶搜索来实现 $O(\log N)$ 的计算复杂度。图12.5展示了基于二分搜索的协同排名函数。协同排名函数使用两对标记变量来描绘 A 数组索引范围和 B 数组索引范围，这些范围被考虑用于生成协同排名值。变量 i 和 j 是当前二分搜索迭代中正在考虑的候选协同排名返回值。变量 i_{low} 和 j_{low} 是函数可能生成的最小协同排名值。第02行将 i 初始化为其最大可能值。如果 k 值大于 m ，第02行将 i 初始化为 m ，因为协同排名 i 值不能大于 A 数组的大小。否则，第02行将 i 初始化为 k ，因为 i 不能大于 k 。协同排名 j 值初始化为 $k - i$ （第03行）。在整个执行过程中，协同排名函数保持这一重要的不变关系。变量 i 和 j 的总和始终等于输入变量 k 的值（排名值）。

i_{low} 和 j_{low} 变量的初始化（第4和5行）需要更多解释。这些变量允许我们限制搜索范围并使其更快。从功能上讲，我们可以将这两个值都设置为零，并让其余执行将它们提升为更准确的值。当 k 值小于 m 和 n 时，这有意

义。然而，当 k 大于 n 时，我们知道 i 值不能小于 $k - n$ 。原因是来自 B 数组的 $C[k]$ 前缀子数组元素的最大数量为 n 。因此，至少 $k - n$ 个元素必须来自 A 。因此， i 值不能小于 $k - n$ ；我们不妨将 i_low 设置为 $k - n$ 。按照同样的逻辑， j_low 值不能小于 $k - m$ ，这也是合并过程中必须使用的 B 元素的最少数量，因此也是最终协同排名 j 值的下限。

我们将使用图12.6中的示例来说明图12.5中的协同排名函数的操作。该示例假设使用三个线程将数组 A 和 B 合并到 C 中。每个线程负责生成一个包含三个元素的输出子数组。我们首先将跟踪线程1的协同排名函数的二分搜索步骤，线程1负责生成 $C[3]$ 到 $C[5]$ 。读者应该能够确定线程1使用参数 $(3, A, 5, B, 4)$ 调用协同排名函数。

如图12.5所示，协同排名函数的第2行将 i 初始化为3，这是 k 值，因为在本例中 k 小于 m （值为5）。此外， i_low 设置为0。 i 和 i_low 值定义了当前正在搜索的 A 数组的部分，以确定最终的协同排名 i 值。因此，只有0、1、2和3被考虑作为协同排名 i 值。类似地， j 和 j_low 值被设置为0和0。

协同排名函数的主体是一个`while`循环（第08行），该循环迭代地缩小到最终的协同排名 i 和 j 值。目标是找到一对 i 和 j 值，使得 $A[i-1] \leq B[j]$ 和 $B[j-1] < A[i]$ 。【译注： $A[i-1] \leq B[j]$ ，这说明 $A[i-1]$ 应该被包含在结果里； $A[i] > B[j-1]$ ，说明 $A[i]$ 不包含在结果了。反过来对 B 也是类似的： $B[j-1] < A[i]$ ，说明 $B[j-1]$ 被包含，而 $B[j] \geq A[i-1]$ ，说明 $B[j]$ 不被包含。】直观上，我们选择 i 和 j 值，以确保用于生成上一个输出子数组的 A 子数组中的任何值都不大于用于生成当前输出子数组的 B 子数组中的任何元素。请注意，由于稳定性要求，当 A 元素和 B 元素之间出现平局时， A 元素在放置到输出数组中时具有优先权，因此上一个子数组中的最大 A 元素可以等于当前 B 子数组中的最小元素。

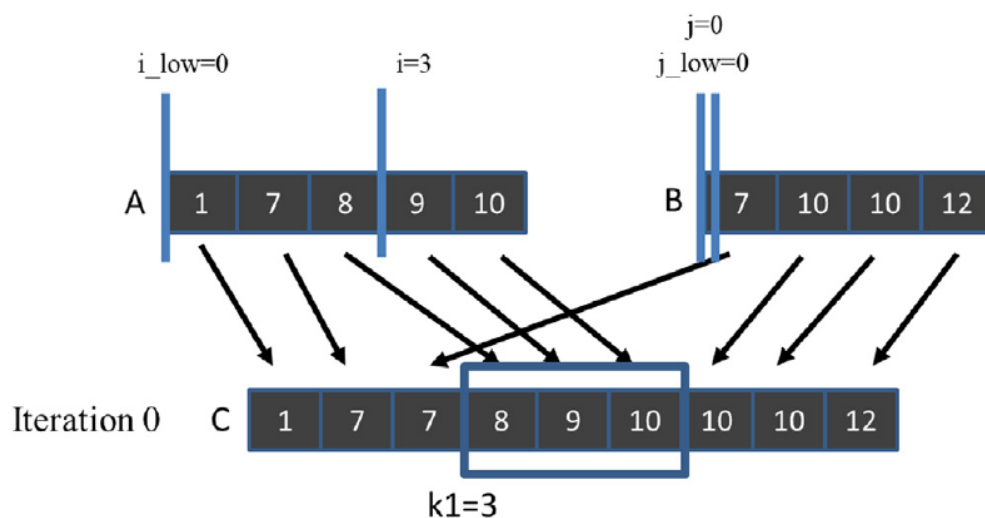


图12.6 线程1的协同排名函数操作示例的第0次迭代。

在图12.5中，`while`循环的第一个`if`结构（第09行）测试当前 i 值是否过高。如果是这样，它将调整标记值，以便将 i 的搜索范围缩小大约一半，向较小的方向移动。这是通过将 i 值减少大约一半的 i 和 i_low 之间的差值来完成的。在图12.7中，对于`while`循环的第0次迭代，`if`结构发现 i 值（3）过高，因为 $A[i-1]$ 的值为8，而 $B[j]$ 的值为7。接下来的几行语句将通过将 i 值减少 $\text{delta} = (3 - 0 + 1) / 2$ （第10和13行）来减少 i 的搜索范围，同时保持 i_low 值不变。因此，下一次迭代的 i_low 和 i 值将为0和1。

代码还将 j 的搜索范围调整为与 i 的搜索范围相当，将其移动到当前 j 位置之上。此调整保持了 i 和 j 的总和等于 k 的属性。调整通过将当前 j 值分配给 j_low （第11行）并将 delta 值加到 j （第12行）来完成。在我们的示例中，下一次迭代的 j_low 和 j 值将为0和2。

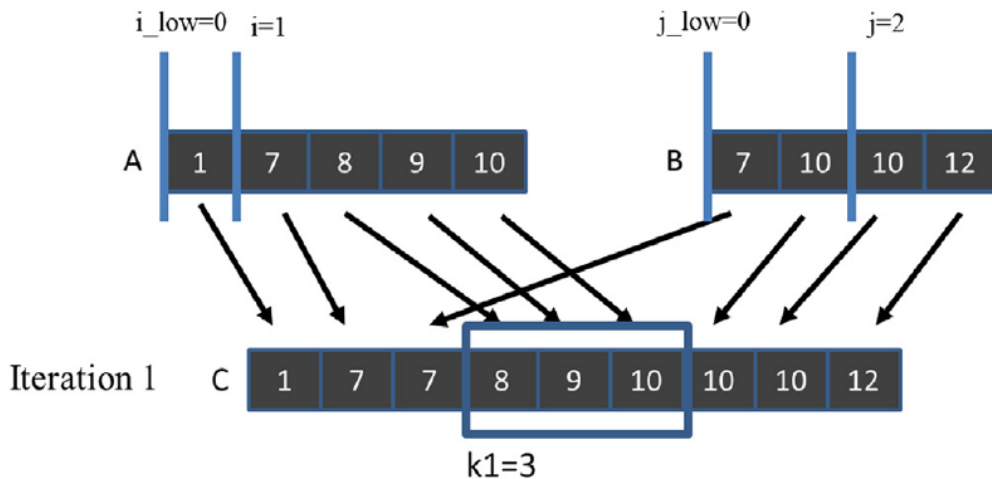


图12.7 线程1的协同排名函数操作示例的第1次迭代。

在while循环的第1次迭代中，如图12.7所示， i 和 j 值分别为1和2。if结构（第9行）发现 i 值是可接受的，因为 $A[i - 1]$ 是 $A[0]$ ，其值为1，而 $B[j]$ 是 $B[2]$ ，其值为10，所以 $A[i - 1]$ 小于 $B[j]$ 。因此，第一个if结构的条件不成立，并且if结构的主体被跳过。然而，在本次迭代中发现 j 值过高，因为 $B[j - 1]$ 是 $B[1]$ （第14行），其值为10，而 $A[i]$ 是 $A[1]$ ，其值为7。因此，第二个if结构将调整标记，以便将 j 的搜索范围减少大约一半，向较低值移动。这是通过从 j 中减去 $\text{delta} = (j - j_{\text{low}} + 1) / 2 = 1$ （第15和18行）来完成的。因此，下一次迭代的 j_{low} 和 j 值将为0和1。它还使 i 的下一个搜索范围与 j 相同大小，但将其向上移动 delta 位置。这是通过将当前 i 值分配给 i_{low} （第16行）并将 delta 值加到 i （第17行）来完成的。因此，下一次迭代的 i_{low} 和 i 值将分别为1和2。

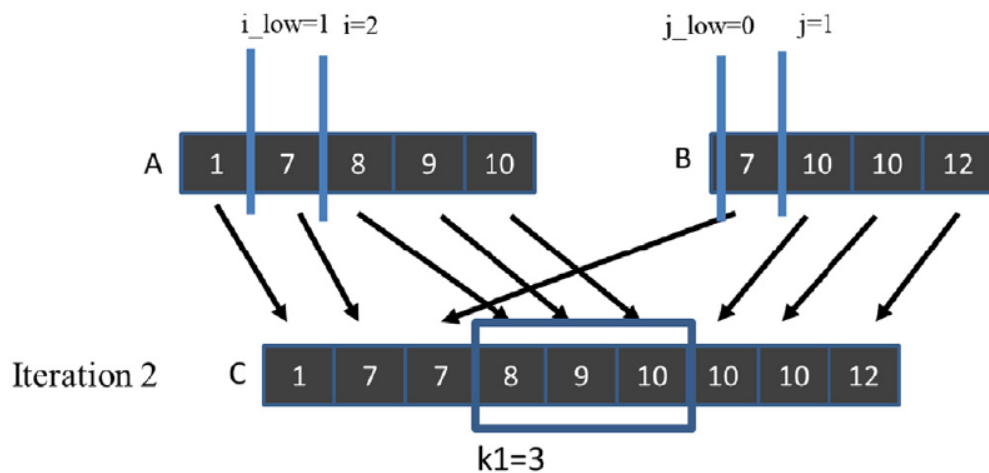


图12.7 线程1的协同排名函数操作示例的第2次迭代。

在第2次迭代中，如图12.8所示， i 和 j 值分别为2和1。两个if结构（第9和14行）都发现 i 和 j 值是可接受的。对于第一个if结构， $A[i - 1]$ 是 $A[1]$ （值为7）和 $B[j]$ 是 $B[1]$ （值为10），因此条件 $A[i - 1] \leq B[j]$ 满足。对于第二个if结构， $B[j - 1]$ 是 $B[0]$ （值为7）和 $A[i]$ 是 $A[2]$ （值为8），因此条件 $B[j - 1] < A[i]$ 也满足。协同排名函数设置一个标志以退出while循环（第20和08行），并返回最终的 i 值2作为协同排名 i 值（第23行）。调用线程可以将最终的协同排名 j 值计算为 $k - i = 3 - 2 = 1$ 。检查图12.8可以确认，协同排名值2和1确实确定了线程1的正确A和B输入子数组。

读者应重复相同的过程来练习线程2。此外，请注意，如果输入流长得多， delta 值将在每一步中减半，因此该算法的复杂度为 $\log_2(N)$ ，其中 N 是两个输入数组大小的最大值。

12.5 基本并行归并内核

在本章的其余部分中，我们假设输入数组 **A** 和 **B** 位于全局内存中。我们进一步假设启动一个内核来合并这两个输入数组，以生成一个也在全局内存中的输出数组 **C**。图 12.9 显示了一个基本的内核，它是第 12.3 节中描述的并行归并函数的直接实现。

```

01 __global__ void merge_basic_kernel(int* A, int m, int* B, int n, int* C) {
02     int tid = blockIdx.x*blockDim.x + threadIdx.x;
03     int elementsPerThread = ceil((m+n)/(blockDim.x*gridDim.x));
04     int k_curr = tid*elementsPerThread; // start output index
05     int k_next = min((tid+1)*elementsPerThread, m+n); // end output index
06     int i_curr = co_rank(k_curr, A, m, B, n);
07     int i_next = co_rank(k_next, A, m, B, n);
08     int j_curr = k_curr - i_curr;
09     int j_next = k_next - i_next;
10     merge_sequential(&A[i_curr], i_next-i_curr, &B[j_curr], j_next-j_curr, &C[k_curr]);
11 }

```

图12.9 一个基本并行归并内核。

正如我们所看到的，内核非常简单。它首先通过计算当前线程要生成的输出子数组的起始点 (k_curr) 和下一个线程的起始点 (k_next) 来划分工作。请记住，输出元素的总数可能不是线程数的倍数。然后，每个线程调用两次 `co_rank` 函数。第一次调用使用 k_curr 作为排名参数，这是当前线程要生成的输出子数组的第一个（最低索引的）元素。返回的 `co_rank` 值 i_curr 给出了属于线程要使用的输入子数组的最低索引的输入 **A** 数组元素。这个 `co_rank` 值也可以用于获取 **B** 输入子数组的 j_curr 。 i_curr 和 j_curr 值标记了线程输入子数组的起点。因此 $\&A[i_curr]$ 和 $\&B[j_curr]$ 是当前线程要使用的输入子数组的起始指针。

第二次调用使用 k_next 作为排名参数来获取下一个线程的 `co_rank` 值。这些 `co_rank` 值标记了下一个线程要使用的最低索引的输入数组元素的位置。因此， $i_next - i_curr$ 和 $j_next - j_curr$ 给出了当前线程要使用的 **A** 和 **B** 子数组的大小。当前线程要生成的输出子数组的起始指针是 $\&C[k_curr]$ 。内核的最后一步是使用这些参数调用 `merge_sequential` 函数（来自图 12.2）。

基本归并内核的执行可以通过图 12.8 中的示例进行说明。三个线程（线程 0、1 和 2）的 k_curr 值分别为 0、3 和 6。我们将重点关注 k_curr 值为 3 的线程 1 的执行。通过第一次 `co_rank` 函数调用确定的 i_curr 和 j_curr 值分别为 2 和 1。线程 1 的 k_next 值将为 6。第二次调用 `co_rank` 函数有助于确定 i_next 和 j_next 值分别为 5 和 1。然后，线程 1 使用参数 ($\&A[2]$, 3, $\&B[1]$, 0, $\&C[3]$) 调用 `merge` 函数。请注意，参数 n 的值为 0 表示线程 1 的输出子数组的三个元素中没有一个是来自数组 **B**。图 12.8 确实如此：输出元素 $C[3]$ 到 $C[5]$ 都来自 $A[2]$ 到 $A[4]$ 。

虽然基本归并内核非常简单和优雅，但它在内存访问效率方面存在不足。首先，很明显在执行 `merge_sequential` 函数时，warp 中相邻线程在读取和写入输入和输出子数组元素时并未访问相邻的内存位置。例如在图 12.8 中的示例中，`merge_sequential` 函数执行的第一次迭代期间，三个相邻线程将读取 $A[0]$ 、 $A[2]$ 和 $B[0]$ 。然后它们将写入 $C[0]$ 、 $C[3]$ 和 $C[6]$ 。因此，它们的内存访问不是合并的，导致内存带宽的利用率很差。

其次，在线程执行 `co_rank` 函数时，它们还需要从全局内存访问 **A** 和 **B** 元素。由于 `co_rank` 函数进行的是二进制搜索，访问模式有些不规则，并且不太可能是合并的。因此，这些访问可能会进一步降低内存带宽利用效率。如果我们能避免 `co_rank` 函数对全局内存的这些非合并访问，那将是有帮助的。

12.6 用于改进内存合并的块状合并内核

在第6章《性能考虑》中，我们提到了改进内核中内存合并(coalescing)的三种主要策略：(1)重新排列线程到数据的映射，(2)重新排列数据本身，以及(3)在全局内存和共享内存之间以合并的方式传输数据，并在共享内存中

执行不规则访问。对于合并模式，我们将使用第三种策略，它利用共享内存来改进合并。使用共享内存还具有在co-rank函数和顺序合并阶段捕捉少量数据重用的优点。

关键的观察点是，相邻线程使用的输入 A 和 B 子数组在内存中是相邻的。本质上，一个块中的所有线程将共同使用更大的、块级别的 A 和 B 子数组来生成更大的、块级别的 C 子数组。我们可以为整个块调用co-rank函数，以获得块级别 A 和 B 子数组的起始和结束位置。使用这些块级别的co-rank值，块中的所有线程可以协作地以合并的模式将块级别的 A 和 B 子数组元素加载到共享内存中。

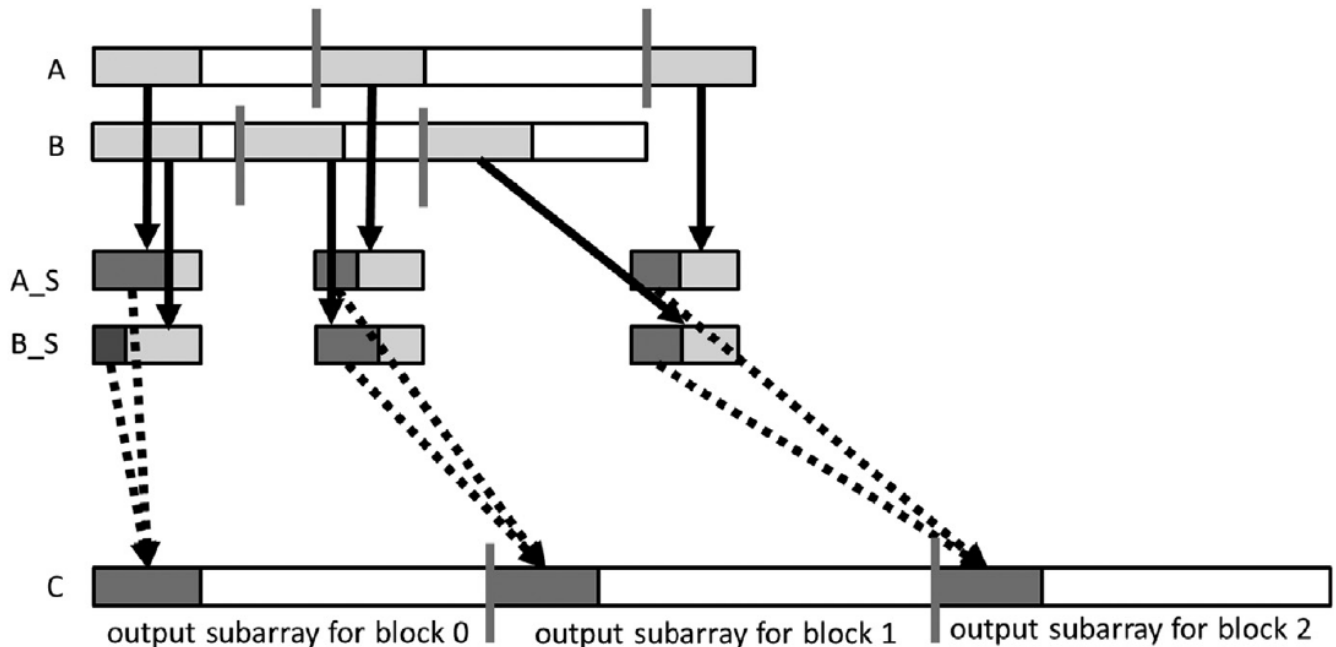


图12.10 块状合并内核的设计。

图12.10显示了一个块状合并内核的块级设计。在这个例子中，我们假设有三个块将用于合并操作。在图的底部，我们展示了 C 被划分为三个块级子数组。我们用灰色竖线来标出这些分区。基于这些分区，每个块调用co-rank函数将输入数组划分为每个块使用的子数组。我们也用灰色竖线标出了输入分区。请注意，根据输入数组中实际数据元素的值，输入分区的大小可能会有显著差异。例如，在图12.8中，线程0的输入 A 子数组明显大于输入 B 子数组。另一方面，线程1的输入 A 子数组明显小于输入 B 子数组。显然，对于每个线程来说，两者输入子数组的总大小必须始终等于输出子数组的大小。

我们将为每个块声明两个共享内存数组 A_S 和 B_S。由于共享内存大小的限制，A_S 和 B_S 可能无法覆盖块的整个输入子数组。因此，我们将采用迭代的方法。假设 A_S 和 B_S 数组每个可以容纳 x 个元素，而每个输出子数组包含 y 个元素。每个线程块将以 y/x 次迭代执行其操作。在每次迭代中，块中的所有线程将协作地从块的输入 A 子数组加载 x 个元素，并从其输入 B 子数组加载 x 个元素。

图12.10展示了每个线程的第一次迭代。我们展示了对于每个块，输入 A 子数组的浅灰色部分被加载到 A_S 中，输入 B 子数组的浅灰色部分被加载到 B_S 中。共享内存中有 x 个 A 元素和 x 个 B 元素，线程块有足够的输入元素来生成至少 x 个输出数组元素。所有线程都保证在迭代过程中拥有它们所需的所有输入子数组元素。可能有人会问，为什么加载总共 $2x$ 个输入元素只能保证生成 x 个输出元素。原因是，在最坏情况下，当前输出部分的所有元素可能都来自一个输入部分。输入使用的不确定性使得合并内核的块状设计比以前的模式更加具有挑战性。可以通过首先调用当前和下一个输出部分的co-rank函数来更准确地加载输入块。在这种情况下，我们需要额外的二分搜索操作以减少冗余数据加载。我们将这种替代实现留作练习。在第12.7节中，我们还将通过循环缓冲区设计来提高内存带宽利用效率。

图 12.10 还显示了每个块中的线程在每次迭代中将使用 A_S 和 B_S 的一部分，如深灰色部分所示，生成输出 C 子数组中的 x 元素部分。这个过程通过虚线箭头从 A_S 和 B_S 的深灰色部分指向 C 的深灰色部分来说明。请注

意，每个线程块可能会使用其 A_S 和 B_S 部分的不同部分。有些块可能更多地使用 A_S 元素，而其他块则可能更多地使用 B_S 元素。每个块实际使用的部分取决于输入数据元素的值。

```

01 __global__ void merge_tiled_kernel(int* A,int m, int* B, int n, int* C, int tile_size) {
    /* shared memory allocation */
02     extern __shared__ int shareAB[];
03     int * A_S = &shareAB[0];  应该是1.0*(m+n)/gridDim.x shareA is first half of shareAB
04     int * B_S = &shareAB[tile_size]; // shareB is second half of shareAB
05     int C_curr = blockIdx.x * ceil((m+n)/gridDim.x); // start point of block's C subarray
06     int C_next = min((blockIdx.x+1) * ceil((m+n)/gridDim.x), (m+n)); // ending point

07     if (threadIdx.x ==0){
08         A_S[0] = co_rank(C_curr, A, m, B, n); // Make block-level co-rank values visible
09         A_S[1] = co_rank(C_next, A, m, B, n); // to other threads in the block
10     }
11     __syncthreads();
12     int A_curr = A_S[0];
13     int A_next = A_S[1];
14     int B_curr = C_curr - A_curr;
15     int B_next = C_next - A_next;
16     __syncthreads();

```

图12.11 第 1 部分：识别块级输出和输入子数组。

【译注：上面的代码(包括后面)存在错误，n/d是整数除法，会舍去小数部分，请参考 <https://stackoverflow.com/questions/26105925/use-of-ceil-and-integers>。】

图 12.11 显示了块状合并内核的第一部分。与图 12.9 相比，显示出显著的相似性。这部分本质上是线程级基本合并内核的设置代码的块级版本。块中只有一个线程需要计算当前块的起始输出索引的秩值和下一个块的起始输出索引的共同秩值。这些值被放置在共享内存中，以便块中的所有线程都可以看到。仅有一个线程调用co-rank函数减少了co-rank函数对全局内存的访问次数，从而提高了全局内存访问的效率。使用一个屏障同步确保所有线程在使用共享内存 A_S[0] 和 A_S[1] 位置的块级共同秩值之前都要等待这些值可用。

请记住，由于输入子数组可能太大而无法放入共享内存，内核采用迭代的方法。内核接收一个 tile_size 参数，该参数指定在共享内存中要容纳的 A 元素和 B 元素的数量。例如，tile_size 值为 1024 意味着在共享内存中要容纳 1024 个 A 数组元素和 1024 个 B 数组元素。这意味着每个块将分配 $(1024 + 1024) * 4 = 8192$ 字节的共享内存来存放 A 和 B 数组元素。

举一个简单的例子，假设我们要合并一个包含 33,000 个元素的 A 数组 ($m=33,000$) 和一个包含 31,000 个元素的 B 数组 ($n=31,000$)。输出 C 元素的总数是 64,000。进一步假设我们将使用 16 个块 ($gridDim.x=16$)，每个块中有 128 个线程 ($blockDim.x=128$)。每个块将生成 $64,000 / 16 = 4000$ 个输出 C 数组元素。

如果假设 tile_size 值为 1024，图 12.12 中的 while 循环需要进行四次迭代才能完成每个块的 4000 个输出元素的生成。在 while 循环的第 0 次迭代期间，每个块中的线程将协作地将 1024 个 A 元素和 1024 个 B 元素加载到共享内存中。由于每个块中有 128 个线程，它们可以在 for 循环（第 26 行）的每次迭代中共同加载 128 个元素。因此，图 12.12 中的第一个 for 循环将迭代 8 次，以便块中的所有线程完成 1024 个 A 元素的加载。第二个 for 循环也将迭代 8 次，以完成 1024 个 B 元素的加载。请注意，线程使用它们的 threadIdx.x 值来选择要加载的元素，因此连续的线程加载连续的元素。内存访问是合并的。我们稍后将回到并解释 if 条件以及加载 A 和 B 元素的索引表达式是如何制定的。

```

17  int counter = 0; //iteration counter
18  int C_length = C_next - C_curr;
19  int A_length = A_next - A_curr;
20  int B_length = B_next - B_curr;
21  int total_iteration = ceil((C_length)/tile_size); //total iteration
22  int C_completed = 0;
23  int A_consumed = 0;
24  int B_consumed = 0;
25  while(counter < total_iteration){
    /* loading tile-size A and B elements into shared memory */
26    for(int i=0; i<tile_size; i+=blockDim.x){
27      if( i + threadIdx.x < A_length - A_consumed) {
28        A_S[i + threadIdx.x] = A[A_curr + A_consumed + i + threadIdx.x ];
29      }
30    }
31    for(int i=0; i<tile_size; i+=blockDim.x) {
32      if(i + threadIdx.x < B_length - B_consumed) {
33        B_S[i + threadIdx.x] = B[B_curr + B_consumed + i + threadIdx.x];
34      }
35    }
36    syncthreads();

```

图12.12 第2部分：将A和B元素加载到共享内存中。

一旦输入数据块被加载到共享内存中，单个线程可以将输入数据块划分并并行合并它们的部分。这是通过为每个线程分配一个输出部分并运行co-rank函数来确定生成该输出部分所需使用的共享内存数据部分来完成的。图12.13中的代码完成了这一步。请记住，这是图12.12中开始的while循环的延续。在while循环的每次迭代中，块中的线程将使用加载到共享内存中的数据总共生成tile_size个C元素（例外情况是最后一次迭代，我们稍后会讨论）。co-rank函数在共享内存中的数据上为每个线程运行。每个线程首先计算其输出范围的起始位置和下一个线程的起始位置，然后使用这些起始位置作为co-rank函数的输入来确定其输入范围。每个线程然后调用顺序合并函数，将其从共享内存中识别出的A和B元素部分（通过共同秩值确定）合并到其指定的C元素范围内。

```

37  int c_curr = threadIdx.x * (tile_size/blockDim.x);
38  int c_next = (threadIdx.x+1) * (tile_size/blockDim.x);
39  c_curr = (c_curr <= C_length - C_completed) ? c_curr : C_length - C_completed;
40  c_next = (c_next <= C_length - C_completed) ? c_next : C_length - C_completed;
41  /* find co-rank for c_curr and c_next */
42  int a_curr = co_rank(c_curr, A_S, min(tile_size, A_length-A_consumed),
43                      B_S, min(tile_size, B_length-B_consumed));
44  int b_curr = c_curr - a_curr;
45  int a_next = co_rank(c_next, A_S, min(tile_size, A_length-A_consumed),
46                      B_S, min(tile_size, B_length-B_consumed));
47  int b_next = c_next - a_next;
48
49  /* All threads call the sequential merge function */
50  merge_sequential (A_S+a_curr, a_next-a_curr, B_S+b_curr, b_next-b_curr,
51                  C+C_curr+C_completed+c_curr);
52  /* Update the number of A and B elements that have been consumed thus far */
53  counter ++;
54  C_completed += tile_size;
55  A_consumed += co_rank(tile_size, A_S, tile_size, B_S, tile_size);
56  B_consumed = C_completed - A_consumed;
57  __syncthreads();
58 }

```

min(tile_size, B_length - B_consumed)

min(tile_size, A_length - A_consumed)

图12.13 第3部分：所有线程并行合并它们各自的子数组。

让我们继续我们的示例。在while循环的每次迭代中，块中的所有线程将共同生成1024个输出元素，使用共享内存中的两个输入数据块的A和B元素（再次强调，我们稍后会处理while循环的最后一次迭代）。工作分配给128个线程，因此每个线程将生成八个输出元素。虽然我们知道每个线程将在共享内存中消耗总共八个输入元素，但我们需要调用co-rank函数来找出每个线程将消耗的A元素和B元素的确切数量及其起始和结束位置。例如，一个线程可能使用三个A元素和五个B元素，而另一个线程可能使用六个A元素和两个B元素，依此类推。

总体而言，在我们的示例中，每次迭代中所有线程在块中使用的 A 元素和 B 元素的总数将加起来为 1024。例如，如果一个块中的所有线程使用了 476 个 A 元素，我们知道它们也使用了 $1024 - 476 = 548$ 个 B 元素。甚至有可能所有线程最终使用 1024 个 A 元素和 0 个 B 元素。请记住，共享内存中加载了总共 2048 个元素。因此在 while 循环的每次迭代中，块中所有线程只会使用加载到共享内存中的一半 A 和 B 元素。

现在我们准备详细研究内核函数的更多细节。回想一下，我们跳过了从全局内存加载 A 和 B 元素到共享内存的索引表达式的解释。对于 while 循环的每次迭代，加载当前数据块的 A 和 B 数组的起始点取决于块中所有线程在之前的 while 循环迭代中消耗的 A 和 B 元素总数。假设我们在变量 `A_consumed` 中记录了 while 循环之前所有迭代中消耗的 A 元素总数。在进入 while 循环之前，我们将 `A_consumed` 初始化为 0。在 while 循环的第 0 次迭代期间，由于在第 0 次迭代开始时 `A_consumed` 为 0，所有块从 `A[A_curr]` 开始其数据块。在 while 循环的每次后续迭代中，A 元素的数据块将从 `A[A_curr + A_consumed]` 开始。

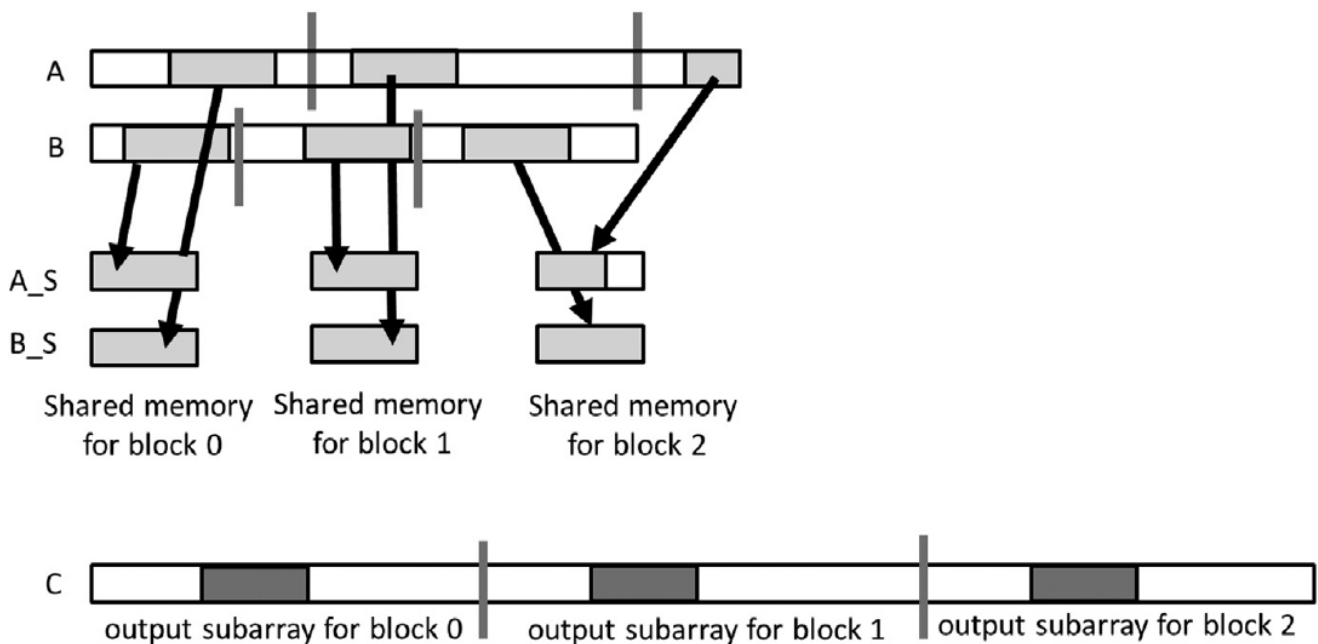


图12.14 运行示例中 while 循环的第 1 次迭代。

图 12.14 说明了 while 循环第 1 次迭代的索引计算。在图 12.10 中，我们展示了在线程块在第 0 次迭代期间消耗的 A_S 元素，这些元素显示为 A_S 中的深灰色部分。在第 1 次迭代中，对于块 0 要从全局内存加载的瓦片应当从包含在第 0 次迭代中消耗的 A 元素部分的下一个位置开始。在图 12.14 中，对于每个块，在第 0 次迭代中消耗的 A 元素部分显示为分配给块的 A 子数组（由竖条标记）的开头的小白色部分。由于小部分的长度由 `A_consumed` 的值给出，因此 while 循环第 1 次迭代要加载的瓦片从 `A[A_curr + A_consumed]` 开始。同样，while 循环第 1 次迭代要加载的瓦片从 `B[B_curr + B_consumed]` 开始。

请注意，在图 12.13 中，`A_consumed`（第 48 行）和 `C_completed` 是在 while 循环迭代中累计的。同样，`B_consumed` 是从累计的 `A_consumed` 和 `C_completed` 值派生的，因此它也在 while 循环迭代中累计。因此，它们始终反映了迄今为止所有迭代消耗的 A 和 B 元素的数量。在每次迭代开始时，要加载的瓦片始终从 `A[A_curr + A_consumed]` 和 `B[B_curr + B_consumed]` 开始。

在 while 循环的最后几次迭代中，某些线程块可能没有足够的输入 A 或 B 元素来填充共享内存中的输入瓦片。例如，在图 12.14 中，对于线程块 2，剩余的 A 元素数量在第 1 次迭代中小于瓦片大小。应使用 if 语句防止线程尝试加载超出块的输入子数组的元素。图 12.12（第 27 行）中的第一个 if 语句通过检查线程尝试加载的 A_S 元素索引是否超过表达式 `A_length - A_consumed` 给出的剩余 A 元素数量来检测此类尝试。if 语句确保线程仅加载 A 子数组剩余部分内的元素。对于 B 元素也是如此（第 32 行）。

使用 if 语句和索引表达式，只要 A_consumed 和 B_consumed 给出线程块在 while 循环前几次迭代中消耗的 A 和 B 元素总数，瓦片加载过程应该正常进行。这使我们进入图 12.13 中 while 循环结束时的代码。这些语句更新了迄今为止 while 循环迭代生成的 C 元素总数。对于除最后一次迭代外的每次迭代，每次迭代生成额外的 tile_size C 元素。

接下来的两个语句更新了块中线程消耗的 A 和 B 元素总数。对于除最后一次迭代外的每次迭代，线程块消耗的额外 A 元素数是返回的值：

```
co_rank(tile_size, A_S, tile_size, B_S, tile_size);
```

【译注：上面的代码是有bug的，应该改为：

```
A_consumed += co_rank(tile_size, A_S, min(tile_size, A_length - A_consumed), B_S,
min(tile_size, B_length - B_consumed));
```

比如A=[0, 1, 4, 5, 5, 7, 8, 9], B=[1, 1, 3, 6, 6, 7, 9]。BLOCK_SIZE=2, GRID_SIZE=2, tile_size=4。那么在第二个块(blockIdx.x=1)的第一个线程(threadIdx.x=0)，它执行的为：

```
co_rank(4, A_S, 4, B_S, 4);
```

其中A_S=[7, 8, 9], B_S=[6, 6, 7, 9]。

但这个时候A_S的长度只剩下3个元素，A_S[3]是未定义的（或者下标越界）。我这里执行时A_S[3]=0，则上面值为4，这是不对的，因为正确值为1。】

正如我们之前提到的，消耗元素数量的计算在 while 循环最后一次迭代结束时可能不正确。最后一次迭代中可能没有剩余完整的瓦片元素。然而，由于 while 循环不再迭代，因此 A_consumed、B_consumed 和 C_completed 值将不再使用，因此不正确的结果不会造成任何问题。然而，应该记住，如果出于任何原因在退出 while 循环后需要这些值，这三个变量将不会具有正确的值。应使用 A_length、B_length 和 C_length 的值，因为在退出 while 循环时，分配给线程块的指定子数组中的所有元素都将被消耗。

这种瓦片内核通过 co-rank 函数实现了全局内存访问的大量减少，并使全局内存访问整合。然而，按原样使用时，内核有一个显著的缺陷。它每次迭代只使用加载到共享内存中的一半数据。共享内存中的未使用数据在下次迭代中被重新加载。这浪费了一半的内存带宽。在下一节中，我们将介绍一种用于管理共享内存中数据元素瓦片的循环缓冲区方案，这使得内核能够充分利用所有加载到共享内存中的 A 和 B 元素。正如我们将看到的，这种增加的效率伴随着代码复杂度的显著增加。

12.7 环形缓冲区合并内核

环形缓冲区(circular buffer)合并内核的设计（以下简称merge_circular_buffer_kernel）与上一节的merge_tiled_kernel内核大致相同。主要区别在于共享内存中A和B元素的管理，以充分利用从全局内存加载的所有元素。merge_tiled_kernel的总体结构如图12.12至12.14所示；假设A和B元素的切片总是从A_S[0]和B_S[0]开始。在每次while循环迭代后，内核加载下一个切片，从A_S[0]和B_S[0]开始。merge_tiled_kernel的低效之处在于

于下一切片的部分元素已经在共享内存中，但我们重新从全局内存加载整个切片，并覆盖这些从上一次迭代中剩余的元素。

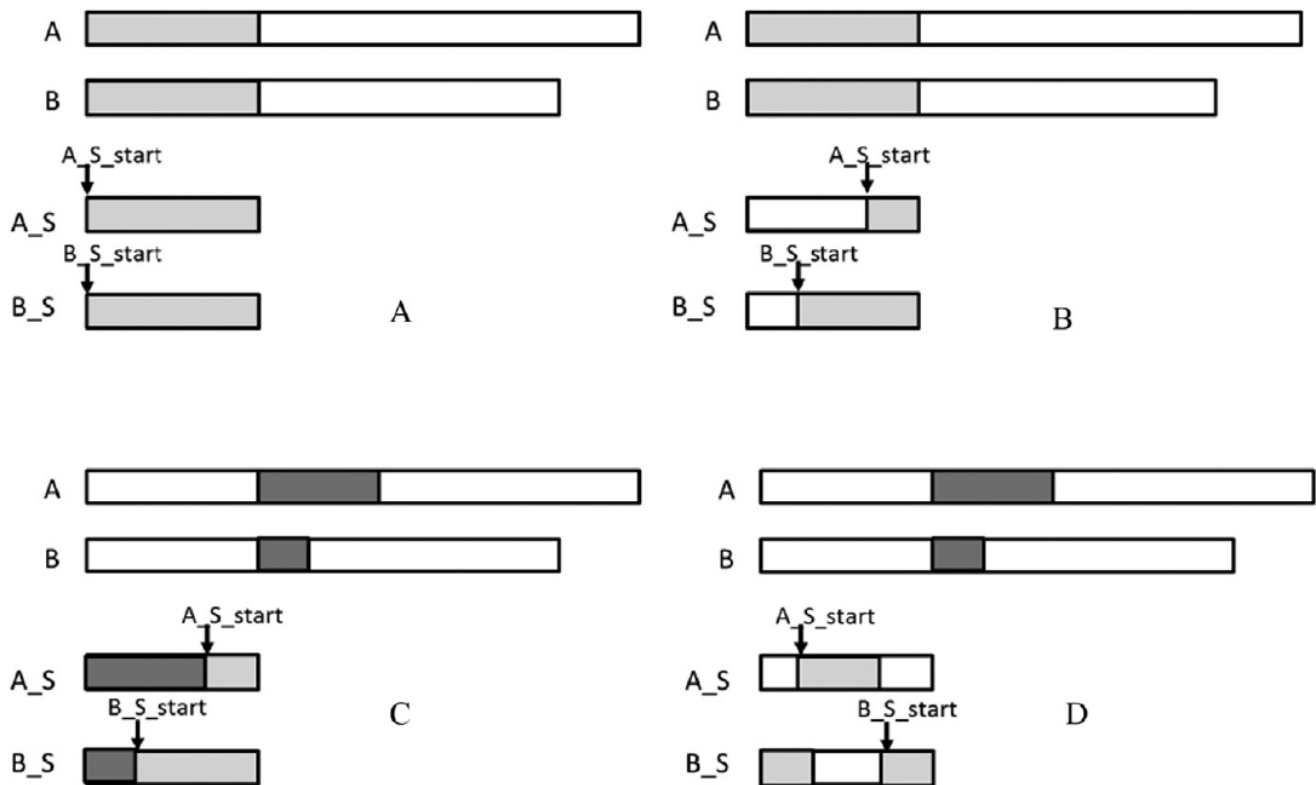


图12.15 环形缓冲区方案用于管理共享内存切片。

图12.15展示了merge_circular_buffer_kernel的主要思想。我们将继续使用图12.10和12.14中的示例。增加了两个变量A_S_start和B_S_start，以允许图12.12中的while循环的每次迭代在动态确定的位置内开始其A和B切片。这种附加跟踪允许每次迭代从上一次迭代中剩余的A和B元素开始切片。由于在首次进入while循环时没有上一次迭代，这两个变量在进入while循环前初始化为0。

在第0次迭代中，由于A_S_start和B_S_start的值都是0，切片将从A_S[0]和B_S[0]开始。图12.15A中显示了从全局内存（A和B）加载到共享内存（A_S和B_S）的切片，这些切片用浅灰色部分表示。一旦这些切片加载到共享内存中，merge_circular_buffer_kernel将以与merge_tiled_kernel相同的方式进行合并操作。

我们还需要通过推进这些变量的值来更新A_S_start和B_S_start变量，以在下一次迭代中使用这些变量。记住每个缓冲区的大小限制为tile_size。在某个时刻，我们需要重新使用A_S和B_S数组的起始部分的缓冲区位置。这可以通过检查新的A_S_start和B_S_start值是否超过tile_size来实现。如果是这样，我们将从它们中减去tile_size，如下if语句所示：

```
A_S_start = (A_S_start + A_S_consumed) % tile_size;
B_S_start = (B_S_start + B_S_consumed) % tile_size;
```

图 12.15B 说明了 A_S_start 和 B_S_start 变量的更新。在迭代 0 结束时，一部分 A 切片和一部分 B 切片已被消耗。被消耗的部分在图 12.15B 的 A_S 和 B_S 中显示为白色部分。我们将 A_S_start 和 B_S_start 的值更新到共享内存中消耗部分之后的位置。

图 12.15C 说明了在 while 循环的迭代 1 开始时填充 A 和 B 切片的操作。A_S_consumed 是一个变量，用于跟踪当前迭代中使用的 A 元素数量。该变量在下一个迭代中填充切片时非常有用。在每次迭代开始时，我们需要

加载一个最多包含 `A_S_consumed` 元素的部分以填充共享内存中的 `A` 切片。同样，我们需要加载一个最多包含 `B_S_consumed` 元素的部分以填充共享内存中的 `B` 切片。加载的这两个部分在图 12.15C 中显示为深灰色部分。请注意，这些切片在 `A_S` 和 `B_S` 数组中实际上是“环绕”的，因为我们在迭代 0 中重新使用了已消耗的 `A` 和 `B` 元素的空间。

图 12.15D 说明了在迭代 1 结束对 `A_S_start` 和 `B_S_start` 的更新。在迭代 1 期间消耗的元素部分显示为白色部分。请注意，在 `A_S` 中，消耗的部分环绕到 `A_S` 的开始部分。`A_S_start` 变量的值也通过 `%` 模运算符进行环绕。显然，我们需要调整加载和使用切片元素的代码，以支持 `A_S` 和 `B_S` 数组的这种循环使用。

`merge_circular_buffer_kernel` 的第 1 部分与图 12.11 中的 `merge_tiled_kernel` 完全相同，因此我们不再展示。图 12.16 显示了循环缓冲区内核的第 2 部分。有关变量声明，请参考图 12.12，它们保持不变。新变量 `A_S_start`、`B_S_start`、`A_S_consumed` 和 `B_S_consumed` 在进入 `while` 循环之前初始化为 0。

```

25 int A_S_start = 0;
26 int B_S_start = 0;
27 int A_S_consumed = tile_size; //in the first iteration, fill the tile_size
28 int B_S_consumed = tile_size; //in the first iteration, fill the tile_size
29 while(counter < total_iteration) {
    /* loading A_S_consumed elements into A_S */
30     for(int i=0; i<A_S_consumed; i+=blockDim.x) {
31         if(i+threadIdx.x < A_length-A_consumed && (i+threadIdx.x) < A_S_consumed) {
32             A_S[(A_S_start + (tile_size - A_S_consumed)+ i + threadIdx.x)%tile_size] =
                A[A_curr + A_consumed + i + threadIdx.x];
33         }
34     }
    /* loading B_S_consumed elements into B_S */
35     for(int i=0; i<B_S_consumed; i+=blockDim.x) {
36         if(i+threadIdx.x < B_length-B_consumed && (i+threadIdx.x) < B_S_consumed) {
37             B_S[(B_S_start + (tile_size - A_S_consumed) + i + threadIdx.x)%tile_size] =
                B[B_curr + B_consumed + i + threadIdx.x];
38         }
39     }

```

图12.16 循环缓冲合并内核的第 2 部分。

请注意，两次 `for` 循环的退出条件已调整。在图 12.12 中的合并内核中，每个 `for` 循环都设置为加载需要重新填充切片的元素数量，即 `A_S_consumed`。在线程块的第 i 次 `for` 循环迭代中要加载的 `A` 元素部分从全局内存位置 `A[A_curr + A_consumed + i]` 开始。请注意，在每次迭代后， i 增加 `blockDim.x`。因此，线程在第 i 次 `for` 循环迭代中要加载的 `A` 元素为 `A[A_curr + A_consumed + i + threadIdx.x]`。每个线程将其 `A` 元素放入 `A_S` 数组的索引为 `A_S_start + (tile_size - A_S_consumed) + i + threadIdx`，因为切片从 `A_S[A_S_start]` 开始，并且 `while` 循环的前一次迭代中缓冲区中剩余 `(tile_size - A_S_consumed)` 个元素。模 `(%)` 运算检查索引值是否大于或等于 `tile_size`。如果是，则通过从索引值中减去 `tile_size`，将其环绕回数组的开始部分。同样的分析适用于加载 `B` 切片的 `for` 循环，并留给读者作为练习。

将 `A_S` 和 `B_S` 数组用作循环缓冲区也会在实际实现 `co-rank` 和 `merge` 函数时带来额外的复杂性。部分额外的复杂性可能反映在调用这些函数的线程级代码中。然而，通常情况下，如果能够在库函数内部有效地处理复杂性，以最小化用户代码中复杂性的增加，将是更好的方法。我们在图 12.17 中展示了这种方法。图 12.17A 显示了循环缓冲区的实现。`A_S_start` 和 `B_S_start` 标记了循环缓冲区中切片的开始位置。切片在 `A_S` 和 `B_S` 数组中环绕，显示为 `A_S_start` 和 `B_S_start` 左侧的浅灰色部分。

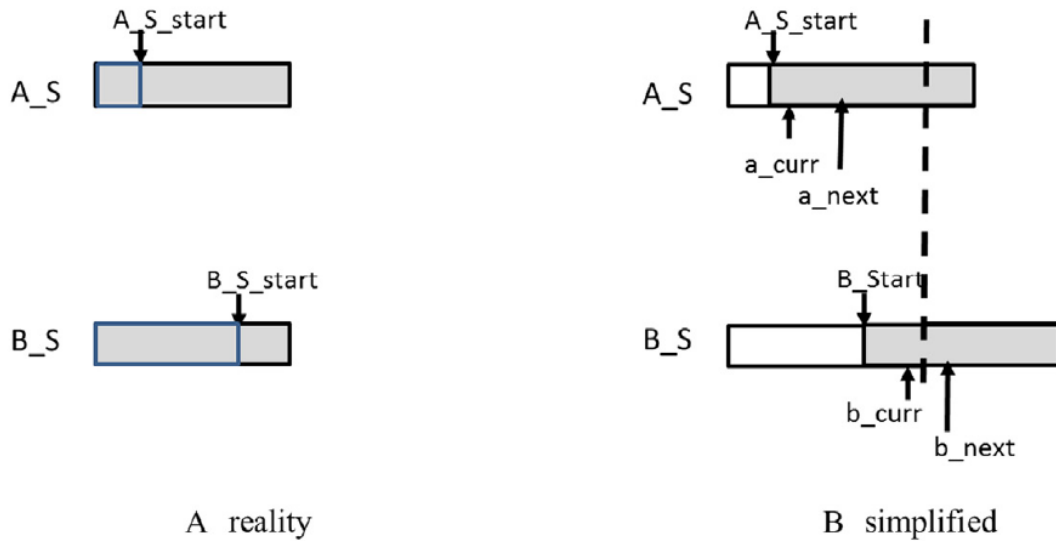


图12.17 使用环形缓冲区时co-rank的简化模型。

记住，co-rank值用于线程标识它们要使用的输入子数组的起始位置、结束位置和长度。当我们使用环形缓冲区时，我们可以将co-rank值提供为环形缓冲区中的实际索引。然而，这将在 `merge_circular_buffer_kernel` 代码中增加相当多的复杂性。例如，`a_next` 值可能小于 `a_curr` 值，因为在 `A_S` 数组中，瓦片已经绕过了。因此，需要测试这种情况并计算部分的长度为 $a_next - a_curr + \text{tile_size}$ 。然而，在其他情况下，当 `a_next` 大于 `a_curr` 时，部分的长度简单地为 $a_next - a_curr$ 。

图 12.17B 展示了使用环形缓冲区定义、导出和使用co-rank值的简化模型。在这个模型中，每个瓦片似乎都在从 `A_S_start` 和 `B_S_start` 开始的连续部分中。在图 12.17A 中的 `B_S` 瓦片的情况下，`b_next` 被绕回并且会小于环形缓冲区中的 `b_curr`。然而，正如图 12.17B 所示，简化模型提供了所有元素都在最多 `tile_size` 元素的连续部分中的错觉；因此 `a_next` 总是大于或等于 `a_curr`，而 `b_next` 总是大于或等于 `b_curr`。将这种对co-rank值的简化视图映射到实际环形缓冲区索引是由 `co_rank_circular` 和 `merge_sequential_circular` 函数的实现来完成的，以便它们能够正确且高效地执行其功能。

```

40     int c_curr = threadIdx.x * (tile_size/blockDim.x);
41     int c_next = (threadIdx.x+1) * (tile_size/blockDim.x);

42     c_curr = (c_curr <= C_length-C_completed) ? c_curr : C_length-C_completed;
43     c_next = (c_next <= C_length-C_completed) ? c_next : C_length-C_completed;
/* find co-rank for c_curr and c_next */
44     int a_curr = co_rank_circular(c_curr,
                                   A_S, min(tile_size, A_length-A_consumed),
                                   B_S, min(tile_size, B_length-B_consumed),
                                   A_S_start, B_S_start, tile_size);
45     int b_curr = c_curr - a_curr;
46     int a_next = co_rank_circular(c_next,
                                   A_S, min(tile_size, A_length-A_consumed),
                                   B_S, min(tile_size, B_length-B_consumed),
                                   A_S_start, B_S_start, tile_size);

47     int b_next = c_next - a_next;
/* All threads call the circular-buffer version of the sequential merge function */
48     merge_sequential_circular( A_S, a_next-a_curr,
                                B_S, b_next-b_curr, C+C_curr+C_completed+c_curr,
                                A_S_start+a_curr, B_S_start+b_curr, tile_size);

/* Figure out the work has been done */
49     counter++;
50     A_S_consumed = co_rank_circular(min(tile_size,C_length-C_completed),
                                     A_S, min(tile_size, A_length-A_consumed),
                                     B_S, min(tile_size, B_length-B_consumed),
                                     A_S_start, B_S_start, tile_size);

51     B_S_consumed = min(tile_size, C_length-C_completed) - A_S_consumed;
52     A_consumed += A_S_consumed;
53     C_completed += min(tile_size, C_length-C_completed);
54     B_consumed = C_completed - A_consumed;

55     A_S_start = (A_S_start + A_S_consumed) % tile_size;
56     B_S_start = (B_S_start + B_S_consumed) % tile_size;
57     __syncthreads();
58 }
59 }

```

图12.18 环形缓冲区合并内核的第三部分。

co_rank_circular 和 merge_sequential_circular 函数与原始的 co_rank 和 merge 函数具有相同的一组参数，再加上三个额外的参数：A_S_start、B_S_start 和 tile_size。这三个额外的参数告诉函数当前缓冲区的起始点在哪里以及缓冲区有多大。图 12.18 展示了基于环形缓冲区的 co-rank 值简化模型的修改后的线程级代码。代码唯一的变化是调用 co_rank_circular 和 merge_sequential_circular 函数而不是 co_rank 和 merge 函数。这表明，一个设计良好的库接口可以减少在使用复杂数据结构时对用户代码的影响。

```

int co_rank_circular(int k, int* A, int m, int* B, int n, int A_S_start, int
B_S_start, int tile_size) {
    int i = k < m ? k : m; // i = min(k,m)
    int j = k - i;
    int i_low = 0 > (k-n) ? 0 : k-n; // i_low = max(0, k-n)
    int j_low = 0 > (k-m) ? 0 : k-m; // i_low = max(0,k-m)
    int delta;
    bool active = true;
    while(active) {
        int i_cir = (A_S_start+i) % tile_size;
        int i_m_1_cir = (A_S_start+i-1) % tile_size;
        int j_cir = (B_S_start+j) % tile_size;
        int j_m_1_cir = (B_S_start+i-1) % tile_size;
        if (i > 0 && j < n && A[i_m_1_cir] > B[j_cir]) {
            delta = ((i - i_low + 1) >> 1) ; // ceil(i-i_low)/2)
            j_low = j;
            i = i - delta;
            j = j + delta;
        } else if (j > 0 && i < m && B[j_m_1_cir] >= A[i_cir]) {
            delta = ((j - j_low + 1) >> 1) ;
            i_low = i;
            i = i + delta;
            j = j - delta;
        } else {
            active = false;
        }
    }
    return i;
}

```

i需要改成j

图12.19 一个在环形缓冲区上运行的co-rank 环形函数。

图 12.19 展示了提供co-rank值的简化模型并正确操作环形缓冲区的 co-rank 函数的实现。它将 i、j、i_low 和 j_low 值的处理方式与图 12.5 中的co-rank函数完全相同。唯一的变化是 i、i - 1、j 和 j - 1 不再直接用作访问 A_S 和 B_S 数组的索引。它们用作要添加到 A_S_start 和 B_S_start 值的偏移量，以形成索引值 i_cir、i_m_1_cir、j_cir 和 j_m_1_cir。在每种情况下，我们需要测试实际索引值是否需要绕回到缓冲区的开始部分。注意，我们不能简单地使用 i_cir - 1 来替换 i - 1。我们需要形成最终的索引值并检查是否需要将其绕回。简化模型有助于保持co-rank函数代码的简洁：所有对 i、j、i_low 和 j_low 值的操作保持不变；它们不需要处理缓冲区的环形特性。

```

void merge_sequential_circular(int *A, int m, int *B, int n, int *C, int
A_S_start, int B_S_start, int tile_size) {
    int i = 0; //virtual index into A
    int j = 0; //virtual index into B
    int k = 0; //virtual index into C
    while ((i < m) && (j < n)) {
        int i_cir = (A_S_start + i) % tile_size;
        int j_cir = (B_S_start + j) % tile_size;
        if (A[i_cir] <= B[j_cir]) {
            C[k++] = A[i_cir]; i++;
        } else {
            C[k++] = B[j_cir]; j++;
        }
    }
    if (i == m) { //done with A[] handle remaining B[]
        for (; j < n; j++) {
            int j_cir = (B_S_start + j) % tile_size;
            C[k++] = B[j_cir];
        }
    } else { //done with B[], handle remaining A[]
        for (; i < m; i++) {
            int i_cir = (A_S_start + i) % tile_size;
            C[k++] = A[i_cir];
        }
    }
}

```

图12.20 `merge_sequential_circular` 函数的实现。

图 12.20 展示了 `merge_sequential_circular` 函数的实现。与 `co_rank_circular` 函数类似，代码逻辑与原始的 `merge` 函数基本保持不变。唯一的变化在于如何使用 `i` 和 `j` 来访问 `A` 和 `B` 元素。由于 `merge_sequential_circular` 函数只会被 `merge_circular_buffer_kernel` 的线程级代码调用，因此访问的 `A` 和 `B` 元素将在 `A_S` 和 `B_S` 数组中。在使用 `i` 或 `j` 访问 `A` 或 `B` 元素的四个地方中，我们需要形成 `i_cir` 或 `j_cir` 并测试索引值是否需要绕回到数组的开始部分。否则，代码与图 12.2 中的 `merge` 函数相同。

尽管我们没有列出 `merge_circular_buffer_kernel` 的所有部分，读者应该能够根据我们讨论的部分将它们整合在一起。使用切片和环形缓冲区增加了相当多的复杂性。特别是，每个线程使用了更多的寄存器来跟踪缓冲区的起始点和剩余元素的数量。所有这些额外的用法潜在地会降低占用率，即在执行内核时可以分配给每个流多处理器的线程块数量。然而，由于合并操作是内存带宽受限的，计算和寄存器资源可能被低效利用。因此，增加使用的寄存器数量和地址计算以节省内存带宽是一个合理的权衡。

【译注：上面的代码是错误的，举个例子。

`A=[0, 1, 4, 5, 5, 7, 8, 9]`, `B=[1, 1, 3, 6, 6, 7, 9]`, `BLOCK_SIZE=2`, `GRID_SIZE=2`, `tile_size=4`。

我们来看第一个块(`blockIdx.x==0`)，在第一次迭代时，`A_S=[0, 1, 4, 5]`，`B_S=[1, 1, 3]`，合并后`C=[0, 1, 1, 1]`，`A_consumed=2`，`B_consumed=2`。也就是说第一次合并产生`C`的前4个值。并且循环数组`A_S`的指针`A_S_start=2`，`B_S`的指针`B_S_start=2`。

接下来在第二次迭代加载`A_S`和`B_S`就出问题了。我们仔细看一下加载的代码：

```

for(int i = 0; i < A_S_consumed; i += blockDim.x){
    if(i + threadIdx.x < A_length - A_consumed && i + threadIdx.x <
A_S_consumed){

```

```

        A_S[(A_S_start + (tile_size - A_S_consumed) + i + threadIdx.x) %
tile_size] = A[A_curr + A_consumed + i + threadIdx.x];
    }
}

```

在第二次迭代时，A_S消费了2个值，因此需要补充两个新的值。我们来看第一个线程(threadIdx.x==0)，在i==0的时候，它是满足条件的：

```

i + threadIdx.x < A_length - A_consumed && i + threadIdx.x < A_S_consumed
0 + 0 < 5 - 2 && 0 + 0 < 2

```

我们再看修改的A_S下标：

```

(A_S_start + (tile_size - A_S_consumed) + i + threadIdx.x) % tile_size
(2 + (4 - 2) + 0 + 0) % 4 = 0

```

也就是修改下标为0的，这也是对的，我们回顾一下A_S目前的情况：

```

0 | 1 | 4 | 5
  |
  /|\
  |
  |
A_S_start的位置

```

如上图所示，A_S_start指向第2个元素(4)，并且4和5是未消费的，因此下一个值(A[4]=5)需要放到第0个位置，这是没有问题的。

问题出在读取A的索引上：

```

A[A_curr + A_consumed + i + threadIdx.x]
A[0 + 2 + 0 + 0] = A[2] = 4

```

可以看到它读取的下标不对。原因是：之前的算法每次都重新加载所有未消费的值，4和5会被重新加载一遍。但是使用了循环数组之后，4和5就不能再被加载了。因此我们需要用两个新的变量记录A和B已经加载到A_S和B_S中的位置，正确的代码是：

```

int A_loaded = 0;
int B_loaded = 0;

while(counter < total_iteration){
    for(int i = 0; i < A_S_consumed; i += blockDim.x){

```

```

        if(i + threadIdx.x < A_length - A_loaded && i + threadIdx.x <
A_S_consumed){
            A_S[(A_S_start + (tile_size - A_S_consumed) + i + threadIdx.x) %
tile_size] = A[A_curr + A_loaded + i + threadIdx.x];
        }
    }
    A_loaded += min(A_S_consumed, A_length - A_loaded);

    for(int i = 0; i < B_S_consumed; i += blockDim.x){
        if(i + threadIdx.x < B_length - B_loaded && i + threadIdx.x <
B_S_consumed){
            B_S[(B_S_start + (tile_size - B_S_consumed) + i + threadIdx.x) %
tile_size] = B[B_curr + B_loaded + i + threadIdx.x];
        }
    }
    B_loaded += min(B_S_consumed, A_length - B_loaded);

```

前面的判断条件从A_length - A_consumed改成了A_length - A_loaded，而读取的下标从A_curr + A_consumed + i + threadIdx.x改成了A_curr + A_loaded + i + threadIdx.x。

另外A_loaded += min(A_S_consumed, A_length - A_loaded)用于更新本次迭代加载到A_S里的数量。

】

12.8 线程粗化合并

将合并并行化到多个线程的代价主要在于每个线程必须执行自己的二分搜索操作来确定其输出索引的co-rank。通过减少启动的线程数量可以减少执行的二分搜索操作的数量，这可以通过为每个线程分配更多的输出元素来实现。本章介绍的所有内核已经应用了线程粗化，因为它们都是编写成每个线程处理多个元素的。在一个完全没有粗化的内核中，每个线程将负责一个单独的输出元素。然而，这将需要为每个单独的元素执行一次二分搜索操作，这将会非常昂贵。因此，粗化对于将二分搜索操作的成本摊销到大量元素上是必不可少的。

12.9 总结

在本章中，我们介绍了有序合并模式，其并行化要求每个线程动态识别其输入位置范围。由于输入范围依赖于数据，我们采用快速搜索实现的联合秩函数来识别每个线程的输入范围。输入范围依赖于数据的事实在我们使用平铺技术来节省内存带宽并实现内存合并时也带来了额外的挑战。因此，我们引入了循环缓冲区的使用，以便充分利用从全局内存加载的数据。我们展示了引入更复杂的数据结构（例如循环缓冲区）可以显著增加使用该数据结构的代码的复杂性。因此，我们为操纵和使用索引的代码引入了一个简化的缓冲区访问模型，使其基本保持不变。实际的缓冲区循环性质仅在这些索引用于访问缓冲区中的元素时才会体现出来。